

1. INTRODUCTION

On travaille dans ce sujet sur des *graphes bipartis*. Un graphe est dit biparti lorsque l'ensemble de ses sommets peut s'écrire sous la forme $V = L \cup R$, où $L \cap R = \emptyset$, et de telle sorte qu'il n'existe aucune arête entre deux sommets de L , ni aucune arête entre deux sommets quelconques de R .

On représentera les graphes par leur listes d'adjacence :

```
type graph = int list array
```

OCAML

IMPORTANT

C'est un sujet où il est impératif de faire des dessins de graphes pour comprendre les définitions présentées.

2. COUPLAGES

Un *couplage* est un ensemble d'arêtes M d'un graphe qui vérifie : si e_1 et e_2 sont deux arêtes de M , alors ces arêtes ne partagent aucun sommet commun. On dit que deux sommets du graphe sont couplés s'ils sont reliés par une arête de M .

Le but est de déterminer un couplage de cardinal maximum.

On représentera un couplage par un tableau de type :

```
type matching = int option array
```

OCAML

Si `matching` est un couplage, alors `matching.(i)` vaut `None` si le sommet i n'est couplé avec aucun sommet. Sinon, `matching.(i)` indique le numéro j du sommet couplé avec i . Dans ce cas, on a nécessairement `matching.(j) = Some i`.

Question 1. Écrire une fonction `empty_matching` qui, à partir d'un graphe biparti, crée le couplage vide (c'est-à-dire le couplage où aucun sommet n'est couplé).

```
val empty_matching: graph -> matching
```

OCAML

Question 2. Écrire une fonction `mate` telle que l'appel `mate matching v1 v2` ajoute l'arête entre v_1 et v_2 au couplage. Cette fonction n'effectue aucune vérification (notamment, elle ne vérifie pas si les deux sommets étaient déjà couplés, ni si l'arête existe effectivement dans le graphe).

```
val mate : matching -> int -> int -> unit
```

OCAML

Question 3. Écrire une fonction `unmate` telle que l'appel `unmate matching v1` retire, si elle est présente dans le couplage, l'arête contenant v_1 . (On n'oubliera pas qu'il faut mettre à jour **deux** cases dans `matching` : celle de v_1 et celle de l'éventuel sommet couplé.)

```
val unmate : matching -> int -> unit
```

OCAML

3. SOMMETS LIBRES

Soit M un couplage d'un graphe biparti $G = (L \cup R, E)$. Un sommet *libre* est un sommet qui n'est couplé avec aucun sommet de G .

Question 4. Écrire une fonction `free_vertices` qui renvoie la liste des sommets libres du graphe.

```
val free_vertices : matching -> int list
```

OCAML

4. CHEMINS AMÉLIORANTS

Soit M un couplage d'un graphe biparti $G = (L \cup R, E)$. Un *chemin améliorant* est une liste de sommets $[v_1; v_2; \dots; v_n]$ telle que :

- v_1 et v_n sont libres ;
- chaque arête $\{v_i, v_{i+1}\}$ est une arête de G ;
- les arêtes de la forme $\{v_{2i+1}, v_{2i+2}\}$ ne sont **pas** dans le couplage M ;
- les arêtes de la forme $\{v_{2i}, v_{2i+1}\}$ sont des arêtes de M .

Un chemin améliorant C peut se représenter en OCaml comme un couplage : il s'agit du couplage où l'on prend les arêtes $\{v_{2i+1}, v_{2i+2}\}$. Les autres arêtes du chemin améliorant se déduisent de M : le sommet qui suit chaque v_{2i} dans le chemin correspond au sommet couplé avec v_{2i} dans M .

Question 5. Écrire une fonction `symdiff` telle que l'appel `symdiff matching path` retire du couplage `matching` tous les sommets couplés dans `path`, puis ajoute dans `matching` les arêtes du couplage `path`.

```
val symdiff : matching -> matching -> unit
```

OCAML

On peut déterminer un chemin améliorant grâce à un parcours en profondeur dans le graphe, mais en suivant systématiquement les arêtes présentes dans le couplage, en partant des sommets libres.

Question 6. Écrire une fonction `augmenting_path` telle que l'appel `augmenting_path g matching path` renvoie `true` exactement quand il existe un chemin améliorant dans le graphe g . Cette fonction stockera ce chemin dans le paramètre `path` (que l'on supposera égal au couplage vide initialement).

```
val augmenting_path : graph -> matching -> matching -> bool
```

OCAML

Le parcours en profondeur pourra être par exemple programmé comme une fonction auxiliaire interne à `augmenting_path`, qui aurait le type :

```
val dfs : int -> int list -> bool
```

OCAML

telle que l'appel `dfs v1 voisins` cherche un chemin améliorant passant par v_1 et renvoie un booléen indiquant si un tel chemin a été trouvé. Elle modifie le tableau `path` durant la construction du chemin améliorant.

5. COUPLAGE MAXIMUM

Question 7. Programmer la fonction `best_matching`.

```
val best_matching : graph -> matching
```

OCAML

6. ALGORITHME DE HOPCROFT-KARP

On veut améliorer la recherche de chemins améliorants de deux façons :

- en cherchant les chemins les plus courts possibles,
- en cherchant plusieurs chemins améliorants indépendants.

Pour cela, on commence par réaliser un *parcours en largeur* à partir des sommets libres d'une des deux composantes du graphe biparti, en alternant arêtes couplées et arêtes non couplées. Ceci permet de calculer, pour chaque sommet du graphe, sa distance à un sommet libre.

Ensuite, on réalise comme précédemment un parcours en profondeur qui alterne arêtes couplées et non couplées, en partant des sommets libres d'une des deux composantes. Le parcours en profondeur suit l'étiquetage du parcours en largeur précédent : on ne peut emprunter une arête que si elle fait passer d'un sommet à distance k à un sommet à distance $k + 1$. On conserve tous les chemins trouvés.

Question 8. Programmer le parcours en largeur, par exemple en utilisant les fonctions du module `Queue`.

Question 9. Modifier la fonction de la partie précédente, de calcul d'un couplage maximum, afin que le parcours en profondeur suive uniquement les arêtes qui font augmenter la distance. On remarquera qu'un ensemble de chemins améliorants disjoints peut se stocker dans un même tableau.

Question 10. Quelle est la complexité du parcours en largeur, suivi du parcours en profondeur, suivi de l'amélioration du couplage ?

Question 11. Peut-on, au cours de l'algorithme, découvrir des chemins améliorants plus courts que ceux trouvés à un moment précédent ?

Question 12. En déduire qu'après $\sqrt{|V|}$ étapes, l'algorithme ne trouve plus qu'au maximum $\sqrt{|V|}$ chemins améliorants.

Question 13. Déterminer la complexité de cet algorithme.

7. COUPLAGE PARFAIT DE POIDS MAXIMUM

Dans cette partie, on s'intéresse à des graphes bipartis ayant les propriétés suivantes :

- ils sont équilibrés, c'est-à-dire que les deux composantes de sommets ont le même cardinal,
- ils sont complets, c'est-à-dire qu'entre deux sommets d'une composante et de l'autre, l'arête est présente.

Un couplage est parfait lorsque tous les sommets sont couplés.

On attribue à chaque arête du graphe, non orienté, des poids positifs. Le poids d'un couplage est la somme des poids de ses arêtes. On cherche désormais à déterminer non plus un couplage de cardinal maximum mais un couplage parfait de poids maximum. On note $w(u, v)$ le poids de l'arête reliant u et v .

On décrit ici l'algorithme hongrois, qui permet de déterminer un couplage de poids maximum. Pour chaque sommet u , on attribue un *potentiel* $l(u)$. L'étiquetage est *valide* lorsque, pour tous sommets u, v , $l(u) + l(v) \geq w(u, v)$.

Question 14. Déterminer un étiquetage valide initial pour un graphe biparti.

La *marge* d'une arête est la quantité $l(u) + l(v) - w(u, v)$.

Le graphe d'égalité G_l est le sous graphe constitué uniquement des arêtes vérifiant $l(u) + l(v) = w(u, v)$. L'algorithme que l'on implémente recherche des chemins améliorants, tant que c'est possible, dans le graphe d'égalité et lorsque ce n'est plus possible, modifie les potentiels puis recommence l'étape précédente.

On modifie un peu l'algorithme de recherche d'un chemin améliorant : cette étape se fait avec un parcours qui crée des chaînes alternantes (entre arêtes non couplées et arêtes couplées). Ce parcours définit donc des *arbres alternants*. Un tel arbre a pour racine un sommet non couplé, chaque branche est un chemin alternant.

Si l'une des branches est un chemin améliorant, on interrompt la construction de l'arbre alternant et on améliore le couplage.

Si on ne trouve aucun chemin améliorant, on définit la *marge* δ de l'arbre comme étant la plus petite marge parmi celles de toutes

ses arêtes. On modifie les potentiels en augmentant de δ tous ceux des nœuds de la même composante bipartie que la racine, et en diminuant de δ ceux de l'autre composante.

Question 15. Montrer que l'arbre alternant reste dans le graphe d'égalité après cette opération. Montrer que la somme de tous les potentiels diminue.

Pour représenter l'arbre alternant, on pourra considérer un tableau qui à chaque nœud associe le numéro du nœud père dans l'arbre, un nœud racine aura pour père -1 .

Question 16. Écrire une fonction qui détermine un couplage de poids maximum.

8. UN PEU PLUS LOIN

Le sujet d'option informatique du concours commun Mines Ponts 2012 aborde les couplages de cardinal maximum. La troisième partie concerne l'algorithme hongrois dans sa version sans poids.

Comment pourriez-vous résoudre le problème 1045 (The Great Wall Game) de l'UVa ? <https://onlinejudge.org/external/10/1045.pdf>

Last updated 2025-10-06 11:29:36 +0200